

Topic 14: Standard Template Library (STL)

14.1 Overview of the STL

- The STL provides ready-made, generic, highly-optimised data structures and algorithms
- Three main components: Containers (store data), Iterators (traverse containers), Algorithms (process data)
- All containers work with all algorithms through the iterator abstraction
- Using STL saves writing and debugging your own implementations

14.2 vector

- Dynamic array -- size grows automatically as elements are added
- Random access in $O(1)$, `push_back` in amortised $O(1)$
- Prefer vector over raw arrays for most use cases

```
#include <vector>
#include <algorithm>
using namespace std;

vector<int> v = {5, 2, 8, 1, 9};
v.push_back(4);
sort(v.begin(), v.end());
for (int x : v) cout << x << " ";    // 1 2 4 5 8 9
```

14.3 map and unordered_map

- `map<K,V>`: sorted key-value store (Red-Black Tree) -- $O(\log n)$ lookup, insert, delete
- `unordered_map<K,V>`: hash table -- $O(1)$ average lookup, insert, delete
- Use `map` when you need sorted order; `unordered_map` for fastest lookups

```
#include <map>
#include <unordered_map>

map<string, int> scores;
scores["Ali"] = 95;
scores["Sara"] = 88;
cout << scores["Ali"] << endl;    // 95

for (auto& [name, score] : scores)
    cout << name << ": " << score << endl;
```

14.4 stack, queue, priority_queue

- `stack<T>`: LIFO -- `push()`, `pop()`, `top()`
- `queue<T>`: FIFO -- `push()`, `pop()`, `front()`
- `priority_queue<T>`: max-heap by default -- `top()` is always the largest element

```
#include <stack>
#include <queue>

stack<int> st;  st.push(1); st.push(2); st.pop();    // top = 1
queue<int> q;   q.push(1);  q.push(2); q.pop();     // front = 2
```

14.5 Iterators and Algorithms

- Iterator: object that points to an element in a container; `++` moves to the next element
- `begin()` / `end()`: first element / one-past-last sentinel
- Common algorithms: `sort`, `find`, `count`, `reverse`, `accumulate`, `binary_search`, `max_element`

```
vector<int> v = {3,1,4,1,5,9,2};
auto it = find(v.begin(), v.end(), 5);
if (it != v.end()) cout << "Found 5 at index " << it - v.begin();
int sum = accumulate(v.begin(), v.end(), 0); // 25
```